

Terraform AWS Practice — Step-by-Step Playbook

The repeatable process for creating any AWS service, documented services, a reusable template, and interview examples.

This document captures the exact, repeatable steps followed to build services in your terraform-aws-practice project. Use Part 1 as the checklist every time you add a service. Part 2 documents the services built so far, each with a real-world example and interview talking points. Part 3 is a blank template to copy for every new service. Part 4 is the troubleshooting log — the real errors hit and fixed, which are some of the most useful interview material you have.

PART 1 — The Repeatable Playbook (Follow This Every Time)

Every service in the project is added with the SAME 10-step process. Memorizing this sequence is more valuable than memorizing any single service, because it's what a real Infrastructure engineer actually does.

1. **Step 1 — Define the goal:** Decide what you're building and why — the service, the region (us-west-2), and the specific configuration goal (e.g., 'a Free-Tier EC2 web server' or 'a versioned, encrypted S3 bucket').
2. **Step 2 — Create the module:** Create a module folder: modules/<service>/ with three files — variables.tf (inputs), main.tf (the resources), outputs.tf (what to expose). Modules never contain a provider block; they inherit it from the root.
3. **Step 3 — Write the resources:** In main.tf, declare the resource(s) with secure, sensible defaults (encryption on, public access blocked, least-privilege). Use data sources (e.g., latest AMI) instead of hardcoding values that go stale.
4. **Step 4 — Wire it into the root:** Add an enable_<service> toggle variable in the root variables.tf, and a module block in the root main.tf that uses count = var.enable_<service> ? 1 : 0 so the service can be switched on/off.
5. **Step 5 — Expose outputs:** Add outputs in the root outputs.tf (using one()) so a disabled module returns null instead of erroring).
6. **Step 6 — Format & validate:** Run terraform fmt (formats code) and terraform validate (checks syntax/consistency) locally before applying anything.
7. **Step 7 — Plan (preview):** Run terraform plan and READ it carefully — it shows exactly what will be created, changed, or destroyed. Never skip this.
8. **Step 8 — Apply (create):** Run terraform apply and confirm with 'yes'. Terraform makes the AWS API calls to create the resources.

9. **Step 9 — Verify:** Verify the resource actually works — via terraform output, the AWS Console, or an aws CLI command (e.g., open the EC2 public IP, or `aws s3 ls` the bucket).
10. **Step 10 — Commit & clean up:** Commit the code to git and push to GitHub. When finished practicing, run `terraform destroy` to avoid charges.

The Commands (Muscle Memory)

```
terraform fmt      # auto-format the code
terraform validate  # catch syntax/logic errors before touching AWS
terraform init      # download providers/modules (first time / new module)
terraform plan      # PREVIEW changes — always read this
terraform apply     # create/update resources (asks 'yes')
terraform output    # show the outputs (IPs, names, ARNs)
terraform destroy   # tear everything down (stops billing)
```

Interview line: *"My workflow is always `fmt`, `validate`, `plan`, `apply` — I never apply without reading the plan first, because the plan is the one chance to catch a mistake before it touches the live account."*

PART 2 — Services Built So Far (with Interview Examples)

2.1 Amazon S3 (Object Storage)

- **What it is:** Object storage for files at unlimited scale. In this project: a versioned, encrypted bucket with all public access blocked.
- **Where it lives:** `modules/s3_bucket/` — bucket + versioning + encryption + public-access-block + ownership controls.

Real-world example: At a food-delivery company, S3 is the data lake: every order/payment/driver event lands as Parquet files in an S3 bucket, organized into bronze (raw), silver (cleaned), and gold (business tables) prefixes, then queried by Athena and Redshift. The bucket you built is exactly that storage foundation.

Interview line: *"I provisioned a secure S3 bucket — versioning for recovery, SSE-S3 encryption at rest, and all public access blocked — as the storage layer of a data lake. It's where the bronze/silver/gold Parquet layers of a pipeline would live."*

- **Common Q:** Why is a bucket name globally unique? Because it's part of a worldwide namespace — which is why I had to append the account ID to avoid a collision.

2.2 Amazon EC2 (Virtual Servers)

- **What it is:** A virtual machine in the cloud. In this project: a Free-Tier t3.micro running Amazon Linux 2023 with a web server, reachable on its public IP.
- **Where it lives:** `modules/ec2_instance/` — instance + security group + auto-looked-up AMI.

Real-world example: EC2 is the general-purpose compute behind countless systems: hosting an application server, running a self-managed database, or acting as a worker node in a data pipeline. Managed services like EMR (Spark) and Redshift actually run on EC2 under the hood — so understanding EC2 explains how those scale.

Interview line: *"I launched a Free-Tier EC2 instance with a security group scoped to SSH and HTTP, an auto-selected latest Amazon Linux 2023 AMI, and a user-data script that bootstraps a web server — so I could verify it end to end by hitting its public IP in a browser."*

- **Common Q:** What's a security group? A stateful virtual firewall controlling inbound/outbound traffic to the instance — I allowed SSH (22) and HTTP (80).

2.3 AWS IAM (Identities & Permissions)

- **What it is:** The permission system: users, groups, roles, and policies that control who can do what. In this project: a separate terraform-iam-lab with data-engineer and analyst personas plus Glue/Lambda service roles.
- **Where it lives:** terraform-iam-lab (can be folded into the practice project as modules/iam/).

Real-world example: Every AWS data service is gated by IAM. A Glue ETL job doesn't get a password — it ASSUMES an IAM role that grants it exactly the S3 and Glue permissions it needs. That's how services get least-privilege access without anyone handing out credentials.

Interview line: *"I set up IAM with least-privilege policies attached to groups, users added to those groups, and service roles for Glue and Lambda to assume — so permissions are managed centrally and services get temporary credentials instead of long-lived keys."*

- **Common Q:** User vs Role? A user is a permanent identity you log in as; a role is assumed temporarily by a service or another account for short-lived credentials.

2.4 Amazon DynamoDB (NoSQL Key-Value Database)

- **What it is:** A fully-managed NoSQL database with single-digit-millisecond latency at any scale. In this project: an 'Orders' table with a partition key (order_id), sort key (order_date), and a Global Secondary Index on customer_id.
- **Where it lives:** modules/dynamodb_table/ — the aws_dynamodb_table resource with keys + a GSI.

Real-world example: DynamoDB is the go-to store for high-throughput operational data with known access patterns — an orders table, a user-session store, a shopping cart, or a real-time leaderboard. In a food-delivery system it holds live order state that the app reads/writes constantly, while the analytical copy of that data flows to S3 for reporting. It's the operational (OLTP) counterpart to the analytical (OLAP) lake.

Interview line: *"I modeled an Orders table with order_id as the partition key and order_date as the sort key, plus a Global Secondary Index on customer_id so I could query a customer's"*

orders — because in DynamoDB you design the table around your access patterns up front, and any query that isn't on the primary key needs an index."

- **Common Q:** Why a GSI? DynamoDB can only efficiently query by the primary key; a Global Secondary Index adds an alternate key (`customer_id`) so a second query pattern ('all orders for a customer') is also fast.
- **Common Q:** Partition key design? Pick a high-cardinality, evenly-distributed key (`order_id`) to avoid a 'hot partition' where one key gets disproportionate traffic and throttles.

2.5 AWS Lambda (Serverless Compute)

- **What it is:** Serverless functions — you supply code, AWS runs it on demand with no server to manage, and you pay only for execution time. In this project: an 'order-processor' Python function that writes an incoming order to the DynamoDB table.
- **Where it lives:** `modules/lambda_function/` — the function + a least-privilege IAM role + code packaged with `archive_file`.

Real-world example: Lambda is the glue of event-driven data pipelines: react to a file landing in S3, process a message off a Kinesis/SQS stream, transform a record and write it to DynamoDB, or back an API endpoint. It shines for short, event-triggered tasks — anything under its 15-minute limit that doesn't need a full server or distributed compute like Glue/EMR.

Interview line: *"I built an order-processing Lambda in Python that writes orders to DynamoDB. Its IAM role is scoped to only that one table and its indexes — least privilege — and Terraform automatically packages the code and creates the table before the function, because the Lambda module consumes the DynamoDB module's outputs."*

- **Common Q:** Lambda vs EC2 vs Glue? Lambda for short event-driven tasks (<15 min, no server to manage); EC2 for long-running general compute you control; Glue/EMR for large distributed Spark processing. Right tool per job.
- **Common Q:** How did the Lambda get permission to write to DynamoDB? It assumes an IAM execution role whose policy allows `dynamodb:PutItem` on that specific table ARN — no keys, just an assumed role.
- **Integration point:** How the two connect: Terraform saw the Lambda module reference the DynamoDB module's `table_arn` output, so it inferred the dependency and created the table first — I never had to specify ordering manually.

2.6 Amazon Athena + AWS Glue Data Catalog (SQL on S3)

- **What it is:** Athena is a serverless SQL engine that queries files sitting in S3; the Glue Data Catalog holds the schema (table definition) that tells Athena how to read those files. In this project: an 'orders' table over the S3 order JSON, queryable with plain SQL.
- **Where it lives:** `modules/analytics/` — a Glue database + table (explicit JSON schema), an Athena workgroup, and a private results bucket.

Real-world example: This is the heart of a data lake: instead of loading data into a database first, analysts run SQL directly against raw/cleaned files in S3 and pay only per query. A company stores billions of events in S3 and lets analysts answer 'revenue by city last month' with Athena — no servers, no ETL-into-a-warehouse step required for exploratory work.

Interview line: *"I catalogued my S3 order files as a Glue table with an explicit JSON schema, then queried them with Athena — so I can run SQL like `SELECT restaurant, SUM(amount) FROM orders GROUP BY restaurant` directly against files in S3, serverless and pay-per-query."*

- **Common Q:** Why does Athena need the Glue Catalog? Athena has no storage of its own — the Catalog supplies the schema (columns, types, location, format) so Athena knows how to interpret the raw S3 files.
- **Common Q:** How do you keep Athena cheap/fast? Partition the data, use columnar Parquet, and filter on the partition column — Athena charges per TB scanned, so scanning less is both cheaper and faster.

2.7 AWS Glue ETL (Spark) — the flagship DE service

- **What it is:** Managed Apache Spark for large-scale ETL. In this project: a job that reads raw order JSON (bronze), deduplicates and type-conforms it, and writes cleaned Parquet partitioned by order_date (silver) — the medallion pattern.
- **Where it lives:** modules/glue_etl/ — a PySpark script uploaded to a data-lake bucket, a Glue job (Glue 4.0), and a least-privilege IAM role.

Real-world example: Glue is the workhorse of batch data engineering on AWS: it runs the Spark jobs that turn messy raw data into clean, query-ready tables at scale — joining, deduplicating, aggregating across millions of rows. It's the cloud-managed equivalent of the PySpark jobs in the food-delivery project, and it appears in almost every AWS data-engineer job description.

Interview line: *"I built a Glue Spark ETL job that transforms raw order JSON into cleaned, partitioned Parquet — deduplicating on order_id, enforcing types, and partitioning by order_date — the exact bronze-to-silver medallion pattern, running on managed Spark instead of a local cluster."*

- **Common Q:** Lambda vs Glue for ETL? Lambda for small, quick, event-driven transforms (<15 min, single machine); Glue when you need distributed Spark for large data or heavy joins/aggregations.
- **Common Q:** Why write Parquet partitioned by date? Columnar Parquet + partition pruning means downstream queries (Athena/Redshift) scan far less data — the single biggest performance/cost lever in a lake.

PART 3 — Reusable Template (Copy This for Every New Service)

For each new service you add, fill this out. Keeping a filled copy per service builds a portfolio you can talk through in interviews.

Service Documentation Template

SERVICE NAME: (e.g., Amazon RDS)
PURPOSE: what it does in one sentence
MODULE PATH: modules/<service>/
KEY RESOURCES: the main aws_* resources declared
IMPORTANT INPUTS: the variables that matter (size, name, type)
SECURE DEFAULTS: encryption? public access? least privilege?
OUTPUTS: what it exposes (endpoint, ARN, IP)
REAL-WORLD USE: where this is used in a real data platform
INTERVIEW LINE: one sentence you'd say to describe building it
COMMON QUESTION: a likely follow-up + your answer
GOTCHAS: anything that tripped you up (see Part 4)
COST NOTE: Free Tier? what to destroy afterward

The 3-Step 'Add a Service' Pattern (from the project README)

11. Create modules/<service>/ with variables.tf, main.tf, outputs.tf (no provider block).
12. Add an enable_<service> variable in the root variables.tf.
13. Add a module block in the root main.tf with count = var.enable_<service> ? 1 : 0, plus any root outputs.

Filled Example A — DynamoDB (copy this style)

SERVICE NAME: Amazon DynamoDB
PURPOSE: NoSQL key-value store for live order data (operational/OLTP).
MODULE PATH: modules/dynamodb_table/
KEY RESOURCES: aws_dynamodb_table (+ a global_secondary_index block)
IMPORTANT INPUTS: table_name, hash_key=order_id, range_key=order_date, gsi_key=customer_id, billing_mode=PAY_PER_REQUEST
SECURE DEFAULTS: encryption at rest on by default (AWS-owned key)
OUTPUTS: table_name, table_arn
REAL-WORLD USE: live order state an app reads/writes constantly; the analytical copy flows to S3 for reporting
INTERVIEW LINE: 'order_id partition key + order_date sort key, with a GSI on customer_id so a customer's orders are also queryable'
COMMON QUESTION: why a GSI? -> the base table only queries by primary key; a GSI adds a second fast query pattern
GOTCHAS: you only declare attributes used as keys; the rest is schemaless. Watch for hot partitions on low-cardinality keys.
COST NOTE: on-demand ~ \$0 for light use; destroy when done

Filled Example B — Lambda (copy this style)

SERVICE NAME: AWS Lambda
PURPOSE: serverless function that writes an incoming order to DynamoDB.
MODULE PATH: modules/lambda_function/
KEY RESOURCES: aws_lambda_function, aws_iam_role, aws_iam_role_policy, archive_file (zips the code), IAM basic-execution attachment
IMPORTANT INPUTS: function_name, runtime=python3.12, handler=index.handler, dynamodb_table_name + dynamodb_table_arn (from the DDB module)
SECURE DEFAULTS: least-privilege role scoped to ONE table ARN + its indexes; CloudWatch logging via AWSLambdaBasicExecutionRole
OUTPUTS: function_name, function_arn, role_arn

REAL-WORLD USE: event-driven glue: react to S3 uploads, stream messages, or API calls; transform + write to a store

INTERVIEW LINE: 'Python order-processor whose role only allows PutItem on that one table; Terraform packaged the code and ordered it after DDB'

COMMON QUESTION: Lambda vs EC2 vs Glue? -> short event tasks vs long-running compute vs large distributed Spark

GOTCHAS: adding the archive provider means re-running terraform init; the build/ zip is gitignored

COST NOTE: generous free tier; destroy when done

Filled Example C — Athena + Glue Catalog

SERVICE NAME: Amazon Athena + AWS Glue Data Catalog

PURPOSE: run SQL directly against S3 order files (serverless).

MODULE PATH: modules/analytics/

KEY RESOURCES: aws_glue_catalog_database, aws_glue_catalog_table (JSON SerDe, explicit columns), aws_athena_workgroup, results bucket

IMPORTANT INPUTS: database_name=orders_db, table_name=orders, data_location=s3://.../ , results_bucket_name (unique)

SECURE DEFAULTS: private encrypted results bucket, public access blocked

OUTPUTS: database_name, table_name, workgroup_name, results_bucket

REAL-WORLD USE: data-lake analytics – query raw/cleaned S3 data with SQL, pay per TB scanned, no warehouse load step

INTERVIEW LINE: 'catalogued S3 JSON as a Glue table, queried with Athena: SELECT restaurant, SUM(amount) FROM orders GROUP BY restaurant'

COMMON QUESTION: why the Catalog? -> Athena has no storage; the Catalog is the schema that tells it how to read the files

GOTCHAS: keep only order files at data_location (or use a prefix); one JSON object per line for the JSON SerDe

COST NOTE: pay per TB scanned; tiny for this data. destroy when done

Filled Example D — Glue ETL (Spark)

SERVICE NAME: AWS Glue ETL (Spark)

PURPOSE: bronze JSON -> silver Parquet (dedup, type-cast, partition).

MODULE PATH: modules/glue_etl/ (+ scripts/glue_job.py)

KEY RESOURCES: aws_glue_job, aws_iam_role (+ AWSGlueServiceRole + S3 policy), aws_s3_bucket (data lake), aws_s3_object (uploads the script)

IMPORTANT INPUTS: job_name, source_location (bronze), datalake_bucket_name, glue_version=4.0, number_of_workers=2 (G.1X)

SECURE DEFAULTS: least-privilege role: read source + read/write data lake only

OUTPUTS: job_name, datalake_bucket, silver_location

REAL-WORLD USE: the workhorse batch-ETL service – clean/join/aggregate large data into query-ready tables (medallion bronze->silver->gold)

INTERVIEW LINE: 'Spark job that dedups on order_id, casts types, and writes Parquet partitioned by order_date – bronze-to-silver on Glue'

COMMON QUESTION: Lambda vs Glue? -> Lambda small/quick/event-driven; Glue for distributed Spark on large data

GOTCHAS: script lives in S3 (filemd5 etag re-uploads on change); awsglue imports only exist in the Glue runtime

COST NOTE: NOT free tier – ~cents per short run (per DPU-hour). run sparingly; destroy when done

PART 4 — Troubleshooting Log (Interview Gold)

These are the REAL errors hit while building this project and how each was fixed. Interviewers love these because they show you can debug, not just follow a tutorial. Each one is a mini-story you can tell.

Error 1 — Invalid credentials (403 InvalidClientTokenId)

- **Symptom:** Terraform couldn't authenticate; the configured access key was invalid/stale.
- **Root cause & fix:** The local ~/.aws/credentials had an old, deleted key. Fixed by creating a fresh access key and running aws configure, then verifying with aws sts get-caller-identity.

Interview line: *"When I hit InvalidClientTokenId, I knew it was an authentication problem, not a Terraform one — so I checked the active credentials with aws sts get-caller-identity and rotated to a valid key."*

Error 2 — EBS volume too small (InvalidBlockDeviceMapping)

- **Symptom:** 'Volume of size 8GB is smaller than snapshot, expect size >= 30GB'.
- **Root cause & fix:** The Amazon Linux 2023 AMI ships with a 30 GB root snapshot, so the volume can't be smaller. Bumped root_volume_size_gb from 8 to 30 (still Free Tier).

Interview line: *"The AMI's base snapshot was 30 GB, so my 8 GB root volume was rejected. I set it to 30 GB, which is also the Free-Tier limit, so it stayed free."*

Error 3 — Instance type not Free-Tier eligible

- **Symptom:** 'The specified instance type is not eligible for Free Tier' for t2.micro.
- **Root cause & fix:** This account/region's Free-Tier x86 type is t3.micro, not t2.micro. Found the right type with: aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true. Switched to t3.micro.

Interview line: *"Rather than guess, I asked AWS directly which instance types were Free-Tier eligible for my account and switched to t3.micro — AWS has moved many accounts off t2."*

Error 4 — Bucket name already taken (409 BucketAlreadyExists)

- **Symptom:** CreateBucket failed with BucketAlreadyExists.
- **Root cause & fix:** S3 bucket names are globally unique across ALL AWS accounts, and the plain name was taken by someone else. Fixed by appending the account ID to the name via a aws_caller_identity data source, making it unique and stable.

Interview line: *"Bucket names are globally unique, so I made mine deterministic-but-unique by appending my account ID with a caller-identity data source — that avoids collisions without a random suffix that changes every run."*

The general debugging method (say this if asked how you troubleshoot)

Interview line: "I read the error to identify which layer owns it — credentials, the resource config, or a service limit — then I query AWS for the ground truth (caller-identity, describe-instance-types) instead of guessing, fix the specific cause, and re-run plan before apply."

PART 5 — The Project Structure (Reference)

```
terraform-aws-practice/
├── versions.tf          # Terraform + AWS provider versions
├── providers.tf         # ONE aws provider (us-west-2) for the whole project
├── variables.tf         # inputs + enable_s3 / enable_ec2 toggles
├── main.tf              # module blocks wiring services together
├── outputs.tf           # bucket name, EC2 IP, etc.
├── terraform.tfvars.example
├── modules/
│   ├── s3_bucket/      # reusable S3 module (variables/main/outputs)
│   ├── ec2_instance/   # reusable EC2 module
│   ├── dynamodb_table/ # Orders table (order_id + order_date + customer_id GSI)
│   ├── lambda_function/ # order-processor Lambda + src/index.py + IAM role
│   ├── analytics/      # Glue Catalog + Athena (SQL on S3)
│   └── glue_etl/        # Glue Spark ETL: bronze JSON -> silver Parquet
```

- **Where it is:** Local: /Users/pramodbalamuri/PycharmProjects/terraform-aws-practice/
- **Backed up:** GitHub: github.com/pramodbalamuri-commits/terraform-aws-practice (private)

How To Describe This Whole Project in an Interview

Interview line: "I built a modular Terraform project on AWS that's a complete little data platform: order JSON lands in S3, which triggers a Lambda that writes to DynamoDB for real-time state; the same data is catalogued in Glue and queryable with Athena SQL; and a Glue Spark ETL job transforms the raw JSON into cleaned, partitioned Parquet — the bronze-to-silver medallion pattern. Every service (S3, EC2, DynamoDB, Lambda, Athena/Glue, Glue ETL) is a reusable module wired into one root with enable/disable toggles, managed in a single state file, and new services follow a repeatable three-step pattern. Along the way I debugged real issues — credential rotation, AMI volume sizing, Free-Tier instance types, globally-unique bucket naming, and adding S3-read permission for an S3-triggered Lambda — which taught me to diagnose by layer and query AWS for ground truth rather than guess."

That single answer covers the four things every DE role asks about: ingestion (S3), real-time processing (Lambda/DynamoDB), batch ETL (Glue Spark), and SQL analytics (Athena) — all as reproducible Infrastructure as Code.

Verify Commands Cheat Sheet (per service)

```
S3:          aws s3 ls | grep <bucket>
EC2:         open http://<public_ip> (or aws ec2 describe-instances ...)
DynamoDB:    aws dynamodb describe-table --table-name Orders
Lambda:      aws lambda invoke --function-name order-processor \
              --payload '{"order_id":"ord-001","order_date":"2026-08-03"},\'
```

```

        "customer_id":"cust-42","amount":29.99}' \
    --cli-binary-format raw-in-base64-out out.json && cat out.json
Then confirm the write landed:
    aws dynamodb get-item --table-name Orders \
        --key '{"order_id":{"S":"ord-001"},"order_date":{"S":"2026-08-03"}}'
Athena:    aws athena start-query-execution \
        --query-string "SELECT * FROM orders_db.orders" \
        --work-group orders-workgroup    (then get-query-results)
Glue ETL:  aws glue start-job-run --job-name orders-bronze-to-silver
    aws glue get-job-runs --job-name orders-bronze-to-silver \
        --query 'JobRuns[0].JobRunState'    (wait for SUCCEEDED)
    aws s3 ls s3://orders-datalake-<acct>/silver/orders/ --recursive

```